UNIT III: Constructors, Destructors and Operator Overloading

Question Bank Answers: Debugging & Pseudo-Code Formulation

Name	Nikitha R
Reg. No	19252421
Course Code & Name	DSA0105 Object Oriented Programming with C++

Contents

- **Section A: Debugging** (5 problems): line numbers and defect classification, root cause analysis, corrected code and dry run with the predicted terminal output.
- **Section B: Pseudo-Code** (5 problems): design overview, structured pseudo-code and a trace table.
- Every corrected C++ program was compiled with g++ (`-Wall -Wextra`) and run; the outputs shown are the real outputs. The buggy originals were compiled (and, where they compile, run) too, so the quoted compiler/runtime messages are real.

SECTION A: DEBUGGING QUESTIONS

Question 1: Copy Constructor Pass-by-Value Infinite Recursion Trap

Topic: Copy Constructors, Parameter Passing Mechanisms

Step 1: Analyse the problem statement

- **Expected:** `DynamicString s2 = s1;` creates an independent copy with its own heap buffer.
- **Actual:** compilation stops at the copy constructor's declaration.

Step 2: Error found (Task 1)

Line	Classification	Defect
Line 17: `DynamicString(DynamicString source)`	Compilation Error	GCC: **"invalid constructor; you probably meant 'DynamicString (const DynamicString&)'"**. A constructor whose first parameter is the class type *by value* is ill-formed.

Step 3: Root cause explanation (Tasks 2 and 3)

- **Why the recursion would be infinite.** Passing an argument by value means the parameter `source` is *initialized as a copy of the argument*. For a class type, that initialization is a call to the copy constructor. So to call `DynamicString(DynamicString source)` the compiler must first copy-construct `source` by calling... the same constructor, which again needs a by-value parameter, and so on without end. No call could ever begin. The C++ standard avoids this by declaring the form ill-formed, which is why the compiler rejects it at the declaration rather than overflowing the stack at run time.
- **Correct signature:** `DynamicString(const DynamicString &source)`.
- **The & (reference):** the parameter becomes an alias of the caller's object, so no copy is made and the recursion is broken.
- **The const:** (1) it promises the constructor will not modify the source, which is correct because copying must not change the original; (2) a non-const reference cannot bind to a temporary (rvalue) or a const object, so without const the copy constructor could not be used for `DynamicString s = DynamicString("x")`, for const objects, or for pass-by-value of function arguments built from temporaries.
- **Deep copy inside the constructor:** the body allocates a *new* buffer and copies the characters, so each object owns its own memory and the destructor's `delete[]` is safe for both. (A complete resource-owning class should also define an assignment operator: the Rule of Three.)

Step 4: Corrected code

```
#include <iostream>
#include <cstring>

class DynamicString {
private:
    char *buffer;
    int length;

public:
    // Parameterized constructor
    DynamicString(const char *str) {
        length = std::strlen(str);
        buffer = new char[length + 1];
        std::strcpy(buffer, str);
    }

    // Copy constructor: parameter is a const reference
    DynamicString(const DynamicString &source) {
        length = source.length;
        buffer = new char[length + 1];          // fresh heap block (deep copy)
        std::strcpy(buffer, source.buffer);
    }

    ~DynamicString() {
        delete[] buffer;
    }

    void print() const {
        std::cout << buffer << std::endl;
    }
};

int main() {
    DynamicString s1("Antigravity C++");
    DynamicString s2 = s1;                       // copy construction
    s2.print();
    return 0;
}
```

Step 5: Dry run and output

Step	Action	State
1	s1("Antigravity C++"): length = 15, allocate 16 chars, strcpy	s1.buffer = block A
2	DynamicString s2 = s1; copy constructor receives s1 by const reference	no recursion
3	length = 15, allocate a new 16-char block B, strcpy from A	s2.buffer = block B (distinct from A)
4	s2.print()	Antigravity C++
5	end of main: destructors run for s2 then s1, each deleting its own block	no double free

```
Antigravity C++
```

Question 2: Shallow Copy & Double-Free Runtime Crash

Topic: Dynamic Constructors, Destructors, Shallow vs. Deep Copy

Step 1: Analyse the problem statement

- **Expected:** pass the buffer to `processAudio` by value, then continue using `track`.
- **Actual:** compiles cleanly; at run time, the program prints a garbage sample and aborts with "free(): double free detected in tcache 2" (confirmed by running the buggy program).

Step 2: Errors found

Line	Classification	Defect
Missing copy constructor (comment line)	Runtime Bug / Undefined Behavior (design defect)	The compiler-synthesized copy constructor copies the `samples` *pointer value* only (shallow copy).
Destructor: `delete[] samples;`	Undefined Behavior (double free)	Two objects own the same block. The second `delete[]` frees already-freed memory.
Line in main: `track.samples[0]` after the call	Undefined Behavior (use after free)	Reads memory that `buf`'s destructor already released (the buggy run printed garbage, 1446520938).

Step 3: Root cause explanation (Tasks 1 and 2)

- **What the synthesized copy constructor does.** It performs a *memberwise copy*: `capacity` is copied (fine) and `samples`, a raw `int*`, is copied as an address. The pointee (the 5 integers) is **not** duplicated. The compiler cannot know what a raw pointer means (owner? observer? array?), so it does the only safe-looking thing: copy the bits.
- **Address trace.** Before the call: `track.samples = A` (block allocated in the constructor). During `processAudio(track)`: `buf.samples = A` (same address, no new allocation, no constructor message printed). After the call: `buf` is destroyed, its destructor runs `delete[] A`; `track.samples` is still A but A is now freed (dangling).
- **Why the crash.** Back in `main`, reading `track.samples[0]` is a use-after-free (garbage). When `main()` ends, `track`'s destructor executes `delete[] A` a second time. The allocator detects that block A is already in its free list and aborts: "free(): double free detected". With some allocators it silently corrupts the heap instead.
- **Fix.** A user-defined **deep copy constructor** allocates a fresh array for the new object and copies element by element, so `buf.samples = B` and `track.samples = A` are different blocks and each destructor frees exactly its own block.

Step 4: Corrected code (deep copy constructor)

```
#include <iostream>

class AudioBuffer {
public:
    int *samples;
    int capacity;

    AudioBuffer(int cap) : capacity(cap) {
        samples = new int[capacity];
        for (int i = 0; i < capacity; ++i) {
            samples[i] = i * 10;
        }
        std::cout << "Buffer allocated at: " << samples << std::endl;
    }

    // Deep copy constructor
    AudioBuffer(const AudioBuffer &other) : capacity(other.capacity) {
        samples = new int[capacity];             // fresh memory for the copy
        for (int i = 0; i < capacity; ++i) {
            samples[i] = other.samples[i];       // element-by-element copy
        }
        std::cout << "Deep copy allocated at: " << samples << std::endl;
    }

    ~AudioBuffer() {
        std::cout << "Deallocating buffer at: " << samples << std::endl;
        delete[] samples;
    }
};

void processAudio(AudioBuffer buf) {             // copy constructor runs here
    std::cout << "Processing sample 0: " << buf.samples[0] << std::endl;
}

int main() {
    AudioBuffer track(5);
    processAudio(track);

    std::cout << "Back in main, sample 0: " << track.samples[0] << std::endl;
    return 0;
}
```

Step 5: Dry run and verified output

Step	Action	Object state
1	AudioBuffer track(5): allocate block A, fill 0, 10, 20, 30, 40, print address	track.samples = A
2	processAudio(track): deep copy constructor allocates block B, copies 5 ints	buf.samples = B
3	print buf.samples[0]	0
4	function ends: ~AudioBuffer for buf frees B	A still valid
5	print track.samples[0]	0 (valid)
6	end of main: ~AudioBuffer for track frees A	all blocks freed once

```
Buffer allocated at: 0x5650de3762b0
Deep copy allocated at: 0x5650de3772e0
Processing sample 0: 0
Deallocating buffer at: 0x5650de3772e0
Back in main, sample 0: 0
Deallocating buffer at: 0x5650de3762b0
```

Two distinct allocations (A, then B) and two safe deallocations (B, then A). The hexadecimal addresses differ from run to run; only the pattern matters.

Question 3: Stream Insertion Operator << Member Function Violation

Topic: Defining Operator Overloading, Member vs. Non-Member/Friend

Step 1: Analyse the problem statement

- **Expected:** `std::cout << f1 << std::endl;` prints `3/4`.
- **Actual:** Line 20 does not compile, while the reversed Line 23 does.

Step 2: Errors found

Line	Classification	Defect
Line 11: `std::ostream& operator<<(std::ostream &os)` as a member	Design / Semantic Defect	A member operator takes the object on its *left*, so it can only be used as `f1 << stream`.
Line 20: `std::cout << f1`	Compilation Error	GCC: "no match for 'operator<<' (operand types are 'std::ostream' and 'Fraction')".
Line 23: `f1 << std::cout`	Compiles, but inverted semantics	Legal only because the member exists; unacceptable usage.

Step 3: Root cause explanation (Tasks 1 and 2)

- **How operator calls are translated.** For a binary operator `a @ b` the compiler considers (1) a member function `a.operator@(b)` and (2) a non-member function `operator@(a, b)`. For a member operator, the **left operand is the object the function is called on**. So `std::cout << f1` would need `std::cout.operator<<(f1)`, i.e. a member of `std::ostream` accepting a `Fraction`. `ostream` is a library class; we cannot add members to it, and it has none for `Fraction`. No non-member `operator<<(ostream&, const Fraction&)` exists either, so there is no match.
- **Why Line 23 compiles.** `f1 << std::cout` is translated to `f1.operator<<(std::cout)`, which matches the member exactly. It prints the fraction and nothing is wrong technically, but the syntax reads backwards, cannot be chained with strings (`std::cout << "x" << f1` still fails), and violates the universal convention that the stream is on the left. It must not appear in production code.
- **Solution.** Overload `operator<<` as a **non-member** with the stream as the first parameter. Because it must read the private `numerator` and `denominator`, declare it a **friend**. It takes `std::ostream&` (non-const: writing changes the stream) and `const Fraction&` (no copy, no modification), and **returns the stream by reference** so calls chain: `(cout << a) << b`.

Step 4: Corrected code

```
#include <iostream>

class Fraction {
private:
    int numerator;
    int denominator;

public:
    Fraction(int n, int d) : numerator(n), denominator(d) {}

    // Non-member friend: left operand is the stream, right operand is the Fraction
    friend std::ostream& operator<<(std::ostream &os, const Fraction &f);
};

std::ostream& operator<<(std::ostream &os, const Fraction &f) {
    os << f.numerator << "/" << f.denominator;
    return os;                                  // returned to allow chaining
}

int main() {
    Fraction f1(3, 4), f2(5, 8);

    std::cout << f1 << std::endl;
    std::cout << "Fractions: " << f1 << " and " << f2 << std::endl;
    return 0;
}
```

Step 5: Dry run and output

Expression	Translated to	Effect
`std::cout << f1`	`operator<<(std::cout, f1)`	prints 3/4, returns cout
`... << std::endl`	member of ostream on the returned cout	newline and flush
`std::cout << "Fractions: " << f1 << " and " << f2`	four chained calls, left to right	Fractions: 3/4 and 5/8

```
3/4
Fractions: 3/4 and 5/8
```

Question 4: Prefix vs. Postfix Increment Signature Inversion

Topic: Overloading Unary Operators (Prefix ++, Postfix ++(int))

Step 1: Analyse the problem statement

- **Expected:** `StepCounter c2 = ++c1;` leaves both c1 and c2 at 11 (prefix returns the *modified* object).
- **Actual:** it compiles but prints c2 = 10 and c1 = 11 (confirmed by running the buggy program): the two operators' signatures and bodies are swapped.

Step 2: Errors found

Line	Classification	Defect
Line 10: `StepCounter operator++()`	Logic Defect	A parameterless `operator++()` is the **prefix** form, yet its body implements postfix behavior (copy old state, increment, return the copy by value).
Line 17: `StepCounter& operator++(int)`	Logic Defect	The `(int)` form is the **postfix** form, yet its body implements prefix behavior (increment, return reference to the object).

Step 3: Root cause explanation (Tasks 1 to 3)

Operator	Canonical signature	Behavior
Prefix `++obj`	`T& operator++()`	increment first, return reference to the same (now modified) object
Postfix `obj++`	`T operator++(int)`	save the old state, increment, return the saved copy by value

- **Why the code misbehaves.** `++c1` calls `operator++()`, which is the student's postfix-style body: it copies the old state (10), increments `c1` to 11, and returns the copy (10). So `c2` = 10.
- **Purpose of the dummy int.** Two overloads of `operator++` cannot differ only in return type, and both take no real arguments. The language therefore defines that the postfix form has one extra, unnamed `int` parameter that is never used; the compiler passes 0 for it when it translates `c++`. Its sole job is to distinguish the two functions during overload resolution.
- **Why prefix returns by reference.** The result of `++x` is the object itself, so returning `*this` as `T&` avoids a copy and keeps it an lvalue (`++(++x)` and `(++x) = value` work as for built-in types).
- **Why postfix returns by value.** The returned value must be the *old* state, which exists only in a local temporary. Returning a reference to that local would be a dangling reference. Postfix is therefore both required to copy and slightly more expensive: prefer `++i` when the old value is not needed.

Step 4: Corrected code

```
#include <iostream>

class StepCounter {
private:
    int steps;

public:
    StepCounter(int s = 0) : steps(s) {}

    // PREFIX: no parameter, increment first, return reference to the modified object
    StepCounter& operator++() {
        ++steps;
        return *this;
    }

    // POSTFIX: dummy int, save old state, increment, return the old copy by value
    StepCounter operator++(int) {
        StepCounter temp = *this;
        ++steps;
        return temp;
    }

    void display() const {
        std::cout << "Steps: " << steps << std::endl;
    }
};

int main() {
    StepCounter c1(10);

    StepCounter c2 = ++c1;      // prefix
    std::cout << "c2 after prefix:  "; c2.display();
    std::cout << "c1 after prefix:  "; c1.display();

    StepCounter c3 = c1++;      // postfix
    std::cout << "c3 after postfix: "; c3.display();
    std::cout << "c1 after postfix: "; c1.display();
    return 0;
}
```

Step 5: Dry run and output

Statement	steps (c1) after	Value returned	Result variable
StepCounter c1(10)	10	-	-
c2 = ++c1 (prefix)	11	reference to c1 (11)	c2 = 11
c3 = c1++ (postfix)	12	temp copy holding 11	c3 = 11

```
c2 after prefix:  Steps: 11
c1 after prefix:  Steps: 11
c3 after postfix: Steps: 11
c1 after postfix: Steps: 12
```

Question 5: Violation of Fundamental Operator Overloading Rules

Topic: Rules for Overloading Operators

Step 1: Analyse the problem statement

- **Expected:** a vector class with equality, addition and exponentiation.
- **Actual:** three declarations break core language rules and stop compilation.

Step 2: Errors found

Line	Classification	Defect (GCC message)
Line 8: `bool operator.(const MathVector&)`	Compilation Error	The member-access operator cannot be overloaded. "expected type-specifier before '.' token".
Line 14: `int operator+(int a, int b)`	Compilation Error	"'int operator+(int, int)' must have an argument of class or enumerated type".
Line 19: `MathVector operator**(MathVector, int)`	Compilation Error	No such operator exists: "expected initializer before '*' token".

Step 3: Root cause explanation (Tasks 1 to 3)

- **Operators that cannot be overloaded** (five): `.` (member access), `.*` (pointer-to-member access), `::` (scope resolution), `?:` (ternary conditional) and `sizeof`.
- **Why `operator.` is prohibited.** The dot is the language's own way of reaching members. If a class could redefine `obj.member`, then writing any member access, including inside the class's own overloaded function, would call the user function again, and there would be no remaining way to name a real member. The other four have operands that are not ordinary expression values: `::` and `sizeof` work on names/types and are resolved at compile time, `.*` is the companion of the dot, and `?:` evaluates only one of its two result operands, a rule a function call (which evaluates all arguments) cannot reproduce.
- **Line 14 rule.** At least one operand of an overloaded operator must be of a **user-defined type** (a class, struct, union or enumeration). Operators on built-in types (`int + int`) have fixed meaning, so allowing `a * b` to be what `a + b` means for plain integers would make every existing program unpredictable.
- **Line 19: inventing operators.** No. Only the existing set of operator tokens (for example `+ - * / % ^ == != < > [] () -> ++ --`) can be overloaded. The lexer reads `**` as two separate `*` tokens, so it is not an operator. Overloading also **cannot change precedence, associativity or arity**: `+` stays a binary operator that binds like addition; a unary-only operator such as `!` stays unary; default arguments are not allowed for them (except `operator()`).

Step 4: Corrected code (legal == and +, exponentiation as a named method)

```
#include <iostream>

class MathVector {
public:
    double x, y;

    MathVector(double a, double b) : x(a), y(b) {}

    // Legal: overloaded equality operator
    bool operator==(const MathVector &other) const {
        return (x == other.x && y == other.y);
    }

    // Legal: overloaded addition on a user-defined type
    MathVector operator+(const MathVector &other) const {
        return MathVector(x + other.x, y + other.y);
    }

    // Exponentiation as an ordinary named member function
    MathVector power(int p) const {
        double rx = 1.0, ry = 1.0;
        for (int i = 0; i < p; ++i) {
            rx *= x;
            ry *= y;
        }
        return MathVector(rx, ry);
    }
};

int main() {
    MathVector v1(2.0, 3.0), v2(2.0, 3.0);

    std::cout << "v1 == v2 : " << (v1 == v2 ? "true" : "false") << std::endl;

    MathVector sum = v1 + v2;
    std::cout << "v1 + v2  : (" << sum.x << ", " << sum.y << ")" << std::endl;

    MathVector sq = v1.power(2);
    std::cout << "v1^2     : (" << sq.x << ", " << sq.y << ")" << std::endl;
    return 0;
}
```

Step 5: Dry run and output

Statement	Evaluation	Result
`v1 == v2`	2.0 == 2.0 and 3.0 == 3.0	true
`v1 + v2`	(2 + 2, 3 + 3)	(4, 6)
`v1.power(2)`	rx = 2 × 2 = 4; ry = 3 × 3 = 9	(4, 9)

```
v1 == v2 : true
v1 + v2  : (4, 6)
v1^2     : (4, 9)
```

SECTION B: PSEUDO-CODE WRITING QUESTIONS

Question 1: Dynamic String Engine with Deep Copy & Overloaded +

Topic: Dynamic Constructors, Deep Copy Constructors, Destructors, Binary +

Design overview

- **Ownership:** each DynStr owns exactly one heap buffer (size charCount + 1 for the terminator) and frees it in its DESTRUCTOR.
- **Deep copy constructor:** allocates a *fresh* buffer and copies item by item, preventing shallow-copy aliasing and double deallocation.
- **OPERATOR +** builds a temporary buffer of lhs.charCount + rhs.charCount + 1 characters, wraps it in a new DynStr and frees the temporary; the result is returned by value (deep copied).
- The right-hand operand is passed by REF (no copy, read only).

Structured pseudo-code

```
CLASS DynStr
  PRIVATE:
    POINTER TO CHARACTER dataPtr
    INTEGER              charCount

  PUBLIC:
    CONSTRUCTOR(STRING sourceString)
      INTEGER i
      charCount <- LENGTH(sourceString)
      dataPtr   <- ALLOCATE ARRAY[charCount + 1] OF CHARACTER
      FOR i <- 0 TO charCount - 1 STEP 1
        dataPtr[i] <- sourceString[i]
      ENDFOR
      dataPtr[charCount] <- NULL_CHAR
    ENDCONSTRUCTOR

    COPY CONSTRUCTOR(REF DynStr other)
      INTEGER i
      charCount <- other.charCount
      dataPtr   <- ALLOCATE ARRAY[charCount + 1] OF CHARACTER    // fresh buffer
      FOR i <- 0 TO charCount STEP 1                            // includes terminator
        dataPtr[i] <- other.dataPtr[i]
      ENDFOR
    ENDCONSTRUCTOR

    DESTRUCTOR()
      IF dataPtr != NULL THEN
        DEALLOCATE dataPtr
        dataPtr <- NULL
      ENDIF
    ENDDESTRUCTOR

    OPERATOR +(REF DynStr rhs) RETURNS DynStr
      INTEGER total, i
      POINTER TO CHARACTER temp
      total <- charCount + rhs.charCount
      temp  <- ALLOCATE ARRAY[total + 1] OF CHARACTER
      FOR i <- 0 TO charCount - 1 STEP 1
        temp[i] <- dataPtr[i]                       // left operand text
      ENDFOR
      FOR i <- 0 TO rhs.charCount - 1 STEP 1
        temp[charCount + i] <- rhs.dataPtr[i]       // right operand text appended
      ENDFOR
      temp[total] <- NULL_CHAR
      DynStr result(temp)                           // constructor copies temp into its own buffer
      DEALLOCATE temp                               // temporary buffer no longer needed
      RETURN result                                 // returned by value (deep copy)
    ENDOPERATOR
```

```
    METHOD Display()
      DISPLAY dataPtr
    ENDMETHOD
ENDCLASS

BEGIN Main
  DynStr strA("Hello ")
  DynStr strB("World!")
  DynStr strC <- strA + strB
  strC.Display()
END Main                                          // destructors: strC, strB, strA
```

Trace: heap allocations for strA = "Hello " and strB = "World!"

Step	Action	Heap effect (block, size)	charCount / contents
1	strA constructor	ALLOCATE H1, 7 chars	6, "Hello " + NULL
2	strB constructor	ALLOCATE H2, 7 chars	6, "World!" + NULL
3	operator+ : total = 6 + 6 = 12	ALLOCATE temp H3, 13 chars	empty
4	copy left operand (i = 0..5)	H3 written	"Hello "
5	append right operand at offset 6; temp[12] = NULL	H3 written	"Hello World!" (12 chars)
6	DynStr result(temp)	ALLOCATE H4, 13 chars; copy 13	result: 12, "Hello World!"
7	DEALLOCATE temp	FREE H3	-
8	RETURN result: copy constructor builds strC (a compiler may elide this copy)	ALLOCATE H5, 13 chars; copy 13	strC: 12, "Hello World!"
9	temporary result destroyed	FREE H4	-
10	strC.Display()	live blocks: H1, H2, H5	prints Hello World!
11	end of Main: destructors in reverse order strC, strB, strA	FREE H5, H2, H1	no leaks, no double free

Question 2: Complex Number System with Constructor Overloading & Operators (+, *, ==)

Topic: Constructor Overloading, Operator Overloading (+, *, ==)

Design overview

- **Three constructors** chosen by argument count: 0 (0 + 0i), 1 (r + 0i), 2 (r + ii).
- **OPERATOR +** adds components; **OPERATOR *** uses (ac - bd) + (ad + bc)i; **OPERATOR ==** compares both parts and returns BOOLEAN.
- Operators take the right operand by REF and RETURN a new Complex; the operands are not modified.
- A private helper formats the sign so that a negative imaginary part prints as "a - bi".

Structured pseudo-code

```
CLASS Complex
  PRIVATE:
    REAL real
    REAL imag

  PUBLIC:
    CONSTRUCTOR()                              // 0 + 0i
      real <- 0.0
      imag <- 0.0
    ENDCONSTRUCTOR

    CONSTRUCTOR(REAL r)                        // r + 0i
      real <- r
      imag <- 0.0
    ENDCONSTRUCTOR

    CONSTRUCTOR(REAL r, REAL i)                // r + ii
      real <- r
      imag <- i
    ENDCONSTRUCTOR

    OPERATOR +(REF Complex other) RETURNS Complex
      Complex sum(real + other.real, imag + other.imag)
      RETURN sum
    ENDOPERATOR

    OPERATOR *(REF Complex other) RETURNS Complex
      REAL newReal, newImag
      newReal <- (real * other.real) - (imag * other.imag)      // ac - bd
      newImag <- (real * other.imag) + (imag * other.real)      // ad + bc
      Complex product(newReal, newImag)
      RETURN product
    ENDOPERATOR

    OPERATOR ==(REF Complex other) RETURNS BOOLEAN
      IF real = other.real AND imag = other.imag THEN
        RETURN TRUE
      ELSE
        RETURN FALSE
      ENDIF
    ENDOPERATOR

    METHOD Display()
      IF imag >= 0 THEN
        DISPLAY real, " + ", imag, "i"
      ELSE
        DISPLAY real, " - ", ABS(imag), "i"
      ENDIF
```

```
    ENDMETHOD
ENDCLASS

BEGIN Main
  Complex c0                                   // default
  Complex c1(3.0, 2.0)
  Complex c2(1.0, 4.0)
  Complex c3 <- c1 * c2
  Complex c4 <- c1 + c2
  c3.Display()
  c4.Display()
  IF c1 == c2 THEN DISPLAY "equal" ELSE DISPLAY "not equal" ENDIF
END Main
```

Trace: C1 = 3 + 2i, C2 = 1 + 4i, so a = 3, b = 2, c = 1, d = 4

Step	Computation	Value
1	a × c	3 × 1 = 3
2	b × d	2 × 4 = 8
3	newReal = ac - bd	3 - 8 = **-5**
4	a × d	3 × 4 = 12
5	b × c	2 × 1 = 2
6	newImag = ad + bc	12 + 2 = **14**
7	Product object	(-5, 14)
8	Display(): imag = 14 >= 0	-5 + 14i
extra	C1 + C2 = (3 + 1) + (2 + 4)i	4 + 6i
extra	C1 == C2: 3 = 1 is false	FALSE

Question 3: Financial Currency Class with Stream I/O & Unary Negation

Topic: Unary Operator Overloading, Friend Stream Operators (<<, >>)

Design overview

- **Single normalization point:** the constructor converts to total cents (dollars × 100 + cents), then splits with integer division and remainder: dollars = total DIV 100, cents = total MOD 100. Because DIV truncates toward zero, a negative total gives negative dollars and negative cents, which keeps the sign consistent.
- **Unary OPERATOR -** takes no parameter and returns a new Currency with both parts negated (the constructor re-normalizes).
- **FRIEND OPERATORs << and >>** are non-members (stream on the left) and return the stream by REF to allow chaining. The output pads cents to two digits and prints a leading minus for debt.
- >> reads dollars and cents, then assigns a freshly normalized object.

Structured pseudo-code

```
CLASS Currency
  PRIVATE:
    INTEGER dollars
    INTEGER cents
    METHOD Normalize(INTEGER d, INTEGER c)
      INTEGER totalCents
      totalCents <- (d * 100) + c
      dollars    <- totalCents DIV 100            // truncates toward zero
      cents      <- totalCents MOD 100            // same sign as totalCents
    ENDMETHOD

  PUBLIC:
    CONSTRUCTOR(INTEGER d <- 0, INTEGER c <- 0)
      Normalize(d, c)
    ENDCONSTRUCTOR

    OPERATOR -() RETURNS Currency
      Currency negated(-dollars, -cents)          // constructor re-normalizes
      RETURN negated
    ENDOPERATOR

    FRIEND OPERATOR <<(REF OUTSTREAM outStream, REF Currency cur) RETURNS REF OUTSTREAM
      INTEGER absD, absC
      absD <- ABS(cur.dollars)
      absC <- ABS(cur.cents)
      IF cur.dollars < 0 OR cur.cents < 0 THEN
        WRITE outStream, "-"
      ENDIF
      WRITE outStream, "$", absD, "."
      IF absC < 10 THEN
        WRITE outStream, "0"                      // two-digit cents
      ENDIF
      WRITE outStream, absC
      RETURN REF outStream
    ENDOPERATOR

    FRIEND OPERATOR >>(REF INSTREAM inStream, REF Currency cur) RETURNS REF INSTREAM
      INTEGER d, c
      READ inStream, d, c
      cur.Normalize(d, c)                         // friend may call the private helper
      RETURN REF inStream
    ENDOPERATOR
ENDCLASS

BEGIN Main
  Currency price
  Currency debt
  DISPLAY "Enter dollars and cents: "
```

```
  READ price                                      // uses >>
  debt <- -price                                  // uses unary -
  DISPLAY "Amount: ", price
  DISPLAY "Negated: ", debt
END Main
```

Trace: input d = 5, c = 275

Step	Operation	Computation	State
1	operator>> reads d, c	d = 5, c = 275	-
2	Normalize: totalCents	5 × 100 + 275 = 775	-
3	dollars, cents	775 DIV 100 = 7; 775 MOD 100 = 75	dollars = 7, cents = 75
4	operator<< (price)	no minus; absC = 75 (two digits already)	prints **$7.75**
5	unary -price: build Currency(-7, -75)	totalCents = -700 + (-75) = -775	-
6	Normalize	-775 DIV 100 = -7; -775 MOD 100 = -75	dollars = -7, cents = -75
7	operator<< (debt)	negative, so write "-"; absD = 7, absC = 75	prints **-$7.75**

Question 4: Polynomial Expression with Overloaded Subscript [] & Destructor

Topic: Dynamic Constructors, Destructors, Overloaded Subscript Operator []

Design overview

- **Storage:** coeffArray[i] holds the coefficient of x to the power i, so an array of size degree + 1 stores the whole polynomial.
- **Rule of Three:** the constructor allocates, the DESTRUCTOR deallocates, and the COPY CONSTRUCTOR deep-copies, so copies never share a buffer.
- **OPERATOR []** returns REF REAL: an alias to the stored coefficient, so `poly[2] <- 5.0` writes into the polynomial. For an invalid index it prints an alert and returns a reference to a private errorSlot (an extra helper attribute) so no memory outside the array is touched.
- **OPERATOR +** creates a result of the larger degree; a missing coefficient of the shorter operand counts as 0.

Structured pseudo-code

```
CLASS Polynomial
  PRIVATE:
    INTEGER degree
    POINTER TO REAL coeffArray                    // size degree + 1
    REAL errorSlot                                // landing spot for bad indices

  PUBLIC:
    CONSTRUCTOR(INTEGER deg)
      INTEGER i
      degree     <- deg
      coeffArray <- ALLOCATE ARRAY[degree + 1] OF REAL
      FOR i <- 0 TO degree STEP 1
        coeffArray[i] <- 0.0
      ENDFOR
      errorSlot <- 0.0
    ENDCONSTRUCTOR

    COPY CONSTRUCTOR(REF Polynomial other)
      INTEGER i
      degree     <- other.degree
      coeffArray <- ALLOCATE ARRAY[degree + 1] OF REAL      // fresh heap array
      FOR i <- 0 TO degree STEP 1
        coeffArray[i] <- other.coeffArray[i]
      ENDFOR
      errorSlot <- 0.0
    ENDCONSTRUCTOR

    DESTRUCTOR()
      DEALLOCATE coeffArray
      coeffArray <- NULL
    ENDDESTRUCTOR

    OPERATOR [](INTEGER index) RETURNS REF REAL
      IF index >= 0 AND index <= degree THEN
        RETURN REF coeffArray[index]
      ELSE
        DISPLAY "ALERT: coefficient index out of range"
        errorSlot <- 0.0
        RETURN REF errorSlot
      ENDIF
    ENDOPERATOR

    OPERATOR +(REF Polynomial other) RETURNS Polynomial
      INTEGER maxDeg, i
      REAL a, b
      IF degree > other.degree THEN
        maxDeg <- degree
      ELSE
```

```
        maxDeg <- other.degree
      ENDIF
      Polynomial result(maxDeg)
      FOR i <- 0 TO maxDeg STEP 1
        a <- 0.0 ; b <- 0.0
        IF i <= degree THEN a <- coeffArray[i] ENDIF
        IF i <= other.degree THEN b <- other.coeffArray[i] ENDIF
        result[i] <- a + b                        // uses operator[] as an lvalue
      ENDFOR
      RETURN result
    ENDOPERATOR
ENDCLASS

BEGIN Main
  Polynomial p1(2), p2(3)
  p1[0] <- 5.0   p1[1] <- 2.0   p1[2] <- 3.0      // 3x^2 + 2x + 5
  p2[0] <- 7.0   p2[2] <- 1.0   p2[3] <- 4.0      // 4x^3 + x^2 + 7  (p2[1] stays 0)
  Polynomial sum <- p1 + p2
END Main
```

Trace: P1 = 3x^2 + 2x + 5 and P2 = 4x^3 + x^2 + 7

Step	i (power)	a = P1 coeff	b = P2 coeff	result[i] = a + b
setup	-	P1 = [5, 2, 3] (degree 2)	P2 = [7, 0, 1, 4] (degree 3)	maxDeg = 3, result = [0, 0, 0, 0]
1	0	5	7	12
2	1	2	0	2
3	2	3	1	4
4	3	0 (i > 2, not stored)	4	4
result	-	-	-	[12, 2, 4, 4] = **4x^3 + 4x^2 + 2x + 12**

Question 5: 2D Vector with Prefix/Postfix Increment & Compound Assignment

Topic: Unary Operators (Prefix vs Postfix ++), Compound Assignment (+=)

Design overview

- **Prefix ++v:** mutate, then RETURN REF THIS (the object itself).
- **Postfix v++:** takes the dummy INTEGER token, copies the current state into snapshot, mutates, and returns the snapshot BY VALUE (the old state).
- **Compound +=:** mutates THIS by adding the right-hand vector and returns REF THIS so `a += b += c` style chains work.
- Constructor defaults make `Vector2D v` equal (0.0, 0.0).

Structured pseudo-code

```
CLASS Vector2D
  PRIVATE:
    REAL x
    REAL y

  PUBLIC:
    CONSTRUCTOR(REAL px <- 0.0, REAL py <- 0.0)
      x <- px
      y <- py
    ENDCONSTRUCTOR

    OPERATOR ++() RETURNS REF Vector2D            // prefix
      x <- x + 1.0
      y <- y + 1.0
      RETURN REF THIS
    ENDOPERATOR

    OPERATOR ++(INTEGER dummy) RETURNS Vector2D   // postfix (dummy token only distinguishes)
      Vector2D snapshot <- THIS                   // copy of the OLD state
      x <- x + 1.0
      y <- y + 1.0
      RETURN snapshot                             // by value
    ENDOPERATOR

    OPERATOR +=(REF Vector2D rhs) RETURNS REF Vector2D
      x <- x + rhs.x
      y <- y + rhs.y
      RETURN REF THIS
    ENDOPERATOR

    METHOD Display()
      DISPLAY "(", x, ", ", y, ")"
    ENDMETHOD
ENDCLASS

BEGIN Main
  Vector2D v1(5.0, 8.0)
  Vector2D v2, v3
  v2 <- ++v1
  v3 <- v1++
  v1 += v2
  v1.Display()  v2.Display()  v3.Display()
END Main
```

Trace: v1 starts at (5.0, 8.0)

Statement	Operation detail	v1	v2	v3
initial	v1 = (5.0, 8.0); v2, v3 = (0.0, 0.0)	(5, 8)	(0, 0)	(0, 0)
v2 <- ++v1	prefix: v1 becomes (6, 9); reference to v1 returned; v2 copies it	(6, 9)	(6, 9)	(0, 0)
v3 <- v1++	postfix: snapshot = (6, 9); v1 becomes (7, 10); snapshot returned; v3 copies it	(7, 10)	(6, 9)	(6, 9)
v1 += v2 (extra)	(7 + 6, 10 + 9)	(13, 19)	(6, 9)	(6, 9)

After the two statements required by the question: **v1 = (7.0, 10.0), v2 = (6.0, 9.0), v3 = (6.0, 9.0)**. Note that v2 and v3 are equal although v3 came from a postfix call: the postfix result is the state *before* its own increment, and v1 had already been incremented once by the prefix statement.